CMSC132: Linked Lists

By Naren Ayinala

May 2026

Contents

1 Why Linked Lists? 2

2 Singly, Doubly, and Circular Lists 2
2.1 Singly linked list . . . 2
2.2 Doubly linked list . . . 2
2.3 Circular linked list . . . 2

3 Building a Singly Linked List in Java 2
3.1 Trace: inserting at the front . . . 3
3.2 Trace: removing a middle node . . . 4

4 Edge Cases You Must Handle 4

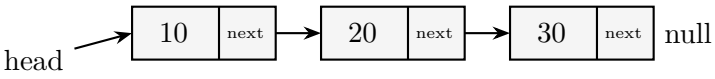
5 Why Index-Based Operations Can Be Slow 4

6 The Java Collections Framework LinkedList 4

7 Practice Questions with Answers 5

1 Why Linked Lists?

An `ArrayList` keeps a resizable array of references. This makes indexed access fast, but inserting near the beginning may shift many elements. A **linked list** instead consists of **nodes**, each holding data and one or more references to other nodes. Nodes do not need to be contiguous in memory.



A **node** is an object. The **head** and **tail** are variables holding references to nodes; they are *not* extra nodes. In an empty list, `head == null`. Keeping a tail reference is optional but often makes appending $O(1)$.

2 Singly, Doubly, and Circular Lists

2.1 Singly linked list

Each node has `data` and `next`. You can follow the chain from the head toward the end, but cannot directly move backward.

```
static class Node<T> {
    T data;
    Node<T> next;
    Node(T data) { this.data = data; }
}
```

2.2 Doubly linked list

Each node has both `next` and `prev`. This supports moving in either direction, but uses extra references and requires updating both directions correctly when nodes are inserted or removed.



2.3 Circular linked list

In a circular singly linked list, the last node's `next` points to the first node rather than null. This can be useful for round-robin scheduling. Traversal must stop after a full cycle; a simple `while (current != null)` would never terminate.

3 Building a Singly Linked List in Java

The following implementation supports insertion at both ends, searching, removing the first matching element, and printing. We keep both head and tail, plus a size counter.

```
import java.util.Objects;
public class BasicLinkedList<T> {
    private static class Node<E> {
        E data;
        Node<E> next;
        Node(E data) { this.data = data; }
    }
    private Node<T> head;
```

```
    private Node<T> tail;
    private int size;

    public int size() { return size; }
    public boolean isEmpty() { return size == 0; }

    public void addFirst(T value) {
        Node<T> node = new Node<>(value);
        node.next = head;
        head = node;
        if (tail == null) tail = node;
        size++;
    }

    public void addLast(T value) {
        Node<T> node = new Node<>(value);
        if (tail == null) head = node;
        else tail.next = node;
        tail = node;
        size++;
    }

    public boolean contains(T value) {
        for (Node<T> cur = head; cur != null; cur = cur.next)
            if (Objects.equals(cur.data, value)) return true;
        return false;
    }

    public boolean removeFirstOccurrence(T value) {
        Node<T> prev = null;
        Node<T> cur = head;
        while (cur != null) {
            if (Objects.equals(cur.data, value)) {
                if (prev == null) head = cur.next;
                else prev.next = cur.next;
                if (cur == tail) tail = prev;
                size--;
                return true;
            }
            prev = cur;
            cur = cur.next;
        }
        return false;
    }

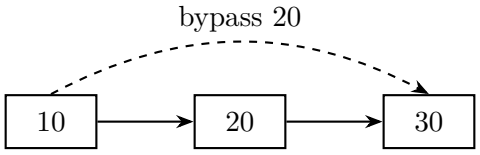
    public void print() {
        for (Node<T> cur = head; cur != null; cur = cur.next)
            System.out.print(cur.data + " -> ");
        System.out.println("null");
    }
}
```

3.1 Trace: inserting at the front

Suppose the list is `20 -> 30 -> null` and we call `addFirst(10)`. First create a new node with data 10. Then set its `next` to the old head (20). Finally move the `head` variable to the new node. The result is `10 -> 20 -> 30 -> null`. **Order matters:** if we overwrite head before saving the old reference, we may lose access to the original chain.

3.2 Trace: removing a middle node

Suppose the list is `10 -> 20 -> 30 -> null`. To remove 20, let `prev` reference 10 and `cur` reference 20. Execute `prev.next = cur.next`. The node holding 10 now references the node holding 30. Once no reachable references point to 20, it can be garbage collected.



4 Edge Cases You Must Handle

1. **Empty list:** inserting the first node must update both head and tail.
2. **Single node:** removing it must set both head and tail to null.
3. **Remove head:** there is no previous node; move head forward.
4. **Remove tail:** update tail to the previous node.
5. **Value absent:** return false without changing the list.
6. **Null data:** use `Objects.equals` if null elements are allowed.

5 Why Index-Based Operations Can Be Slow

To retrieve index 7 in a singly linked list, you start at head and follow seven next references. There is no constant-time jump to the eighth node. This makes `get(i)` an $O(n)$ operation in general, unlike `ArrayList.get(i)` which is $O(1)$.

Operation	ArrayList	Singly linked list with head and tail
Get element by index	$O(1)$	$O(n)$
Add at front	$O(n)$	$O(1)$
Append	$O(1)$ amortized	$O(1)$
Search by value	$O(n)$	$O(n)$
Remove after known node	$O(n)$ shifting in general	$O(1)$ relinking
Remove by value	$O(n)$	$O(n)$

Remember: Linked lists do not magically make *every* insertion $O(1)$. If you must first find the insertion position, locating it can cost $O(n)$. The advantage is that the actual relinking is constant-time once you have the appropriate node references.

6 The Java Collections Framework LinkedList

Java already provides `java.util.LinkedList<E>`, implemented as a doubly linked list. It implements both `List` and `Deque`, so it supports indexed-list operations as well as queue/deque operations.

```
import java.util.*;
LinkedList<String> tasks = new LinkedList<>();
tasks.addFirst("compile");
tasks.addLast("test");
System.out.println(tasks.peekFirst()); // compile
tasks.removeFirst();
System.out.println(tasks); // [test]
```

For most random-access workloads, an `ArrayList` is usually preferable. For a FIFO queue, `ArrayDeque` is often a good default. Use a linked list when its specific insertion/removal behavior is useful.

7 Practice Questions with Answers

Q1. Is the head a node? **Answer:** No, it is a reference to a node.

Q2. What happens when the last element is removed? **Answer:** Head and tail both become null; size becomes 0.

Q3. Why is indexed access $O(n)$? **Answer:** Nodes must be followed sequentially from the head (or from an end in a doubly linked implementation).

Q4. What must change when appending to a nonempty singly linked list? **Answer:** Old tail's next reference, the tail reference, and size.

Q5. Why is a circular list different? **Answer:** The last node links back to the head rather than null, so traversal needs an explicit stopping condition.